Y1T1MID

🗒️ FUNDAMENTAL

FUNDAMENTAL

Precedence	Operator	Description	Order
Highest	()	Parentheses	Left to right
.	**	Exponentiation	Right to left
.	+ -	Unary plus, unary minus	Left to right
.	* / // %	Multiplication, division, modulus	Left to right
Lowest	+ -	Addition, subtraction	Left to right

Types	Classes	Description	Example
Numeric	int, float, complex	Hold numeric values	1, 1.5, 1j
Text	str	Hold sequence of characters	"Hello", 'ABC'
Sequence	list, tuple, range	Hold collection of items	[1,2,1]
Mapping	dict	Hold data in key-value pair form	{"a":"ant"}
Set	set, frozenset	Hold collection of unique items	{1,2,3}
Boolean	bool	Hold Boolean values	True, False
Binary	Bytes, bytearray	Hold byte values	b"ABC"
None	NoneType	Indicate no value	None

non-volatile storage: does not need power to keep data
compile(translate(high level lang) for computer) STATIC-faster *vs* interpret(execute action by action) DYNAMIC-flexible
procedural-oriented prog *vs* object-oriented prog
type conversion: implicit(อ้อม pyแก้ให้อัตโนมัติ//for คแม่นยำ) explicit(ตรง)

F-Strings and Placeholders

- **F-string** allows to format selected parts of a string using **placeholders** {}
- A placeholder can contain variables, operations, functions, and modifiers to format the value

```
width,height = 2,5
area = f"The area is {width*height}"
print(area)  # Output: The area is 10
```

```
price = 50
print(f"The total is {price*1.07} baht")
              # Output: The total is 53.5 baht
```

Number Formatting

- format data directly inside the curly braces by adding a colon (:) followed by a format specifier

```
pi = 3.14159265
price = 1250000
ratio = 0.756

# Fix decimal places (Float)
print(f"Pi: {pi:.2f}")        # Output: Pi: 3.14

# Add comma separators
print(f"Cost: ${price:,}")    # Output: Cost: $1,250,000

# Combined: Commas and decimals
print(f"Cost: ${price:,.2f}") # Output: Cost: $1,250,000.00

# Percentage formatting
print(f"Rate: {ratio:.1%}")   # Output: Rate: 75.6%
```

Padding and Alignment

- Use alignment characters (<, >, ^) along with an integer to set minimum column widths

```
# Aligning text
name = "Alice"
print(f"|{name:<10}|")  # Output: |Alice     |
print(f"|{name:>10}|")  # Output: |     Alice|
print(f"|{name:^10}|")  # Output: |  Alice    |

# Padding with zero instead of spaces
num = 42
print(f"num:05d")       # Output: 00042
```

\n
sep=""
end=""



FUNCTION

```
import math
# value = math.cos(math.pi/2)
# print(value)

# from math import cos, pi
# value = cos(pi/2)

''' //this is docstring
function concept -> sequence of action
    builtin func == anything w x()
        some are ready to use, some needs to be imported
    user defined func == the 'def'

    math.degrees(radian)  == u get degree // same w math.radians(degree)
    math. w sin cos tan uses radian
    math. output mostly came out as float**
        void func = func w no return
            [RULES:
            1.position
            2.u can give func a default value  // def func(x="give
default value here"):]
        variable scope
            local variable == inside func == can be access only inside
that func
            global variable == can be accessed anywhere
            enclosing == คือ global ของ local
            nonlocal ==???
            [-HOW PY SEARCHES- LEGB RULE: L(local)E(enclosing)
G(global)B(built-in)]
            **help()**  if used w our func it will show docstring
    anonymous (lambda) func
        // lambda parameters : expression //
'''

def circle_area(radius:int)->int:
    area= math.pi*(radius**2)
    return area

print(circle_area(20))

square_volume = lambda length : length**3
print(square_volume(30))

x=circle_area(53.4)
print(x)
```

QUIZ1:

```
import math
def read_point(name):
    print(f"Point {name}")
    x = int(input("x: "))
    y = int(input("y: "))
    return(x,y)

def distance(x1,x2,y1,y2):
    dis=(((x2-x1)**2)+((y2-y1)**2))**(1/2)
    return(dis)

x1,y1 = read_point("A")
print()
x2,y2 = read_point("B")

print()
x3,y3 = read_point("C")

print()
line_AB = distance(x1,x2,y1,y2)
line_BC = distance(x2,x3,y2,y3)
line_AC = distance(x1,x3,y1,y3)

perimeter = line_AB + line_BC + line_AC
print(f"Perimeter = {perimeter:.2f}")
```

QUIZ2:

global local enclosing

Exercise 1

```
1: x = 5
2:
3: def test():
4:     x = 8
5:     print(x)
6:
7: test()
8: print(x)
```

What is the output?

8
5

Exercise 2

```
count = 1

def change():
    global count
    count += 2

change()

print(count)
```

What is the output?

3

Exercise 3

```
def outer():

    x = 10

    def inner():
        print(x)

    inner()

outer()
```

Which scope does `x` come from?

Your answer should be one of the following: local, global, enclosing, or built-in.

enclosing

Exercise 4

```
name = "Alice"

def greet():
    print(name)

greet()
```

Is `name` a local or global variable?

Your answer should be either local or global.

global

Exercise 5

```
x = 5

def outer():
```

```
 4:     x = 1
 5:
 6:     def inner():
 7:         nonlocal x
 8:         x += 2
 9:
10:     inner()
11:     print(x)
12:
13: outer()
```

What is the output?

3

`max(0, x)` compares the number `0` and a variable or number `x`.

How `max(0, x)` Works

- If `x` is positive (e.g., `5`): `max(0, 5)` returns `5`.
- If `x` is negative (e.g., `-3`): `max(0, -3)` returns `0`.
- If `x` is zero: `max(0, 0)` returns `0`.

IF ELSE

nested condition [condition inside condition]
chained condition [choose one from many choices]

// biggest to smallest condition vs smallest to biggest condition

LOOP

for we know the iteration times
while we dont know so it repeats until false

zero loop never start bc the condition was not made from the start
infinite loop never stop condition always true
nested loop loop inside loop
Sentinel-controlled loop when u dont know the sequence = can not use for The stop condition is controlled by a *sentinel*.

break out of loop entirely
continue skip the current loop and start from the top again

str = iterable obj
range(start,stop,step)

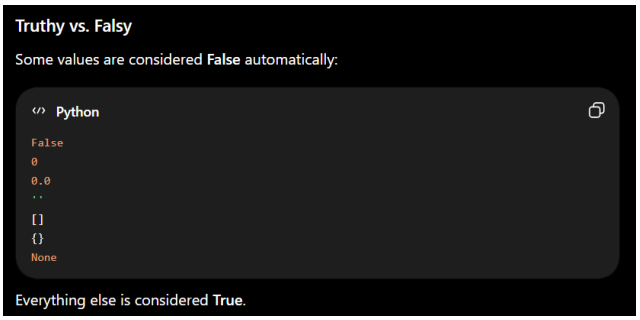
x -= 3; x = x - 3

```python
def avg(num_list):
    n=len(num_list)
    sum_all=0
    for i in range(len(num_list)):
        sum_all+=num_list[i]
    return sum_all/n


num_list=[]
num=input("Enter a number (just [Enter] to quit): ")

if num == "":
    print("Nothing to do.")
else:
    num=float(num)
    num_list.append(num)

while num :
    num=input("Enter a number (just [Enter] to quit): ")
    if not num:
        print(f"The maximum is {max(num_list):.2f}")
        print(f"The minimum is {min(num_list):.2f}")
        print(f"The average is {avg(num_list):.2f}")
        break
    else:
        num=float(num)
        num_list.append(num)
```



the **while num:** here check if num is truthy or falsy

Data Structure	Syntax	Ordered?	Mutable?	Allows Duplicates?
List	`[a, b, c]`	Yes	Yes (Mutable)	Yes
Tuple	`(a, b, c)`	Yes	No (Immutable)	Yes
Dictionary	`{'key': 'value'}`	Yes (Python 3.7+)	Yes (Mutable)	Keys: No / Values: Yes
Set	`{a, b, c}`	No	Yes (Mutable)	No

```
dirr=["n","ne","e","se","s","sw","w","nw"]
current_index=""
angle=0
x,y=("right",angle)

def turn_to(current,target):
    current_index=dirr.index(current)
    print(f"{current} current_index: {current_index}")
    target_index=dirr.index(target)
    print(f"{target} target_index: {target_index}")
    diff_angle=target_index-current_index
    print(diff_angle)
    if diff_angle == 1 or diff_angle == -7:
        print("right 45")
    elif diff_angle ==2 or diff_angle == -6:
        print("right 90")
    elif diff_angle == 3 or diff_angle == -5:
        print("right 135")
    elif diff_angle == 4 or diff_angle == -4:
        print("right 180")
    elif diff_angle ==5 or diff_angle == -3:
        print("left 135")
    elif diff_angle ==6 or diff_angle == -2:
        print("left 90")
    elif diff_angle == 7 or diff_angle == -1:
        print("left 45")
    else:
        print("dont turn")
```

📂 STRINGS, LIST, FILE PROCESSING

String

str[index] it is immutable
len(str)
str traversal = process of visiting each character to do smth
slicing = extract specific part from str str_obj[start:stop:step]

```
text = "Python"

print(text[1:4:1])  # Output: yth
print(text[1:4:2])  # Output: yh
print(text[1:4])    # Output: yth
print(text[:3])     # Output: Pyt
print(text[3:])     # Output: hon
print(text[:])      # Output: Python
print(text[::2])    # Output: Pto
print(text[::-1])   # Output: nohtyP
```

Python provides the **in** and **not in** operators to check whether a string contains another string

```
text = "Python Programming"

print("Python" in text)    # Output: True
print("Java" in text)      # Output: False
print(" " in text)         # Output: True
print("Java" not in text) # Output: True
print(not "Java" in text) # Output: True
```

Function vs. Method (มี “. ”)

Method	Description	Example
upper()	Convert to uppercase	`text = "Python"` `print(text.upper()) # PYTHON`
lower()	Convert to lowercase	`text = "Python"` `print(text.lower()) # python`
strip()	Remove extra spaces	`text = " Python "` `print(text.strip()) # Python`

split()	Split into a list	`text = "Python Programming"`
		`print(text.split(" "))`
		`# ['Python', 'Programming']`

join()	Combine a list into a string	`words = ["ant","boy","cat"]`
		`print("-".join(words))`
		`# ant-boy-cat`

Character	Description	Example
\'	Single quote	`print('He said, \'love Python\'')`
\"	Double quote	`print("He said, \"love Python\"")`
\\	Backslash itself	`print("C:\\Users\\Desktop")`
\n	New line	`print("Hello\nWorld")`
\t	Tab	`print("Hello\tWorld")`

List

list.sort()

Method	Description
append()	Always adds the item to the very end of the list
insert()	Adds the item at a specific position (index)
remove()	Search for the first occurrence of a specific value and deletes it
pop()	Remove and return the item at the specified index
clear()	Remove all elements, leaving the list completely empty
sort()	Sort the list items **in-place**
reverse()	Reverse the current order of the elements **in-place**
count()	Count and return how many times a specific value appears
index()	Return the index position of the first matching item

```
'''
fruits= ['apple','banana','orange','mango']
print(fruits)
fruits.sort()
print(fruits)
fruits.sort(key=len) # sorted(fruits,key=len)
fruits.sort(key=lambda x: x[1]) # sort by second character
print(fruits)
print("mango" in fruits)
print("grape" not in fruits)
#for i in len(fruits):
'''

numbers = [x for x in range(0,11,2)] # list comprehension
print(numbers)
```

```
print([x for x in range(0,11,2)]) #ฉบับคนขี้เกียจ
```

```
numbers = [1,2,4,7]
even = [numbers.index(x) for x in numbers if x%2==0]
# even = [i for i in range(len(numbers)) if numbers[i]%2==0]
print(even)
###
numbers = [5,2,8,4]
print(numbers[::-1])
```

```
# numbers.sort(reverse= True)
print(sorted(numbers,reverse=True))
#print(sum(numbers)/len(numbers)
print( sum(1 for i in numbers if i%2!=0))
```

Files

Opening Files

- Python uses the built-in `open()` function

```
file = open(filename, mode)
```

- The file modes could be:

Mode	Description
'r'	Opens a file for reading text (The file must already exist)
'w'	Opens a file for writing (Overwrites the existing file)
'a'	Opens a file for adding data to the end
'r+'	Open for reading and writing (The file must already exist)
'w+'	Open for reading and writing (Overwrites the existing file)
'a+'	Open for reading and appending (Data is written at the end)

dont forget to close file
file.close()

A better way is to use a context manager

```
with open(filename, mode) as file:
    statements
    ...
```

Advantages:
- Automatically closes the file
- Cleaner code
- Less change of forgetting `close()`

Reading an Entire File

- Python offers different methods depending on how you want to handle the output:
  - `read()` method — Reads everything as a **single large string**
  - `readlines()` method — Reads everything into a **list of strings** (line by line)
  - Looping over file object — The most memory-efficient way

Reading a CSV File

students.csv

```
Alice,20,Bangkok
Bob,22,Chiang Mai
Charlie,19,Phuket
```

```
1:  with open("students.csv") as f:
2:      l = f.read().splitlines()
3:              # ['Alice,20,Bangkok',
4:              #  'Bob,22,Chiang Mai',
5:              #  'Charlie,19,Phuket']
6:
7:      data = [x.split(",") for x in l]
8:              # [['Alice', '20', 'Bangkok'],
9:              #  ['Bob', '22', 'Chiang Mai'],
10:             #  ['Charlie', '19', 'Phuket']]
```

Replace line 7 with this statement:

```
7:      data = [x.split(",") for x in l if x != ""]
```

eval() is python function that take the str inside it and execute

CONCERNING TOPICS_MID

☐ time

How long ago?

Write a program that takes the number of seconds passed (จำนวนวินาทีที่ผ่านไป) and displays the same amount of time in hours, minutes, and seconds.

Example1

```
How many seconds? 378
That is equal to 0 hour(s) 6 minute(s) and 18 second(s).
```

Example 2

```
How many seconds? 8000
That is equal to 2 hour(s) 13 minute(s) and 20 second(s).
```

Hint

Use the operators `//` (integer division) and `%` (remainder after division).

```
seconds= int(input("How many seconds? "))
hrs= seconds//3600
secs= seconds%60
mins= seconds//60

print(f"That is equal to {hrs} hour(s) {mins} minute(s) and
```

[Waiting] เมื่อไรจะได้พบเธอ

เมื่อไรจะได้พบเธอ... เวลา x นาฬิกา เขายังคงนั่งรอเธออยู่ที่สวนสาธารณะเช่นเดิมจนถึงเวลาที่จะได้พบเธอ หากเขาตั้งใจว่าเขาจะรออีก y ชั่วโมงแล้วจะได้พบเธอ อยากทราบว่าเธอจะมาถึงที่สวนสาธารณะเวลากี่โมง

หมายเหตุ x เป็นจำนวนเต็มที่ $0 \leq x < 24$

Example 1

```
x: 14↵
y: 51↵
She will come at 17:00
```

Example 2

```
x: 20↵
y: 37↵
She will come at 9:00
```

```
x=int(input("x: "))
y=int(input("y: "))
time=(x+y)%24
print(f"She will come at {time}:00")
```

Elapsed Time

Write a Python program that accepts a **start time** and a **finish time**, both occurring on the same day, and calculates the elapsed time between them.

Implement the following functions:

- `read_hour()` – returns the hour in 24-hour format (valid values: 0–23).
- `read_minute()` – returns the minute (valid values: 0–59).
- `read_second()` – returns the second (valid values: 0–59).
- `to_seconds(h,m,s)` – returns the total number of seconds represented by `h` hours, `m` minutes, and `s` seconds.
- `elapsed_time(start,finish)` – returns the elapsed time, in seconds, from `start` to `finish`.

Hint:

For example, if the start time is 08:15:00 and the finish time is 13:20:00, you can calculate the elapsed time by first converting both times into the total number of seconds since 00:00:00, and then subtracting the start time (in seconds) from the finish time (in seconds).

Example

```
Start time:
>> Enter hour: 8↵
>> Enter minute: 15↵
>> Enter second: 0↵
Finish time:
>> Enter hour: 13↵
>> Enter minute: 20↵
>> Enter second: 0↵
Elapsed time is 5 hours 5 minutes 0 seconds.
```

```python
total_seconds=0
hour=0
minute=0
second=0
def to_seconds(h,m,s):
    total_seconds=s+(m*60)+(h*3600)
    return total_seconds
def read_hour():
    hour=int(input("Enter hour: "))
    return hour
def read_minute():
    minute=int(input("Enter minute: "))
    return minute
def read_second():
    second=int(input("Enter second: "))
    return second
## get a total_sec
def elapsed_time(start_time,finish_time):
    total_seconds=abs(start_time-finish_time)

    return f"{total_seconds//3600} hours {(total_seconds%3600)//60}
```

```
minutes {(total_seconds%3600)%60} seconds."
```

```
        def read_time():
            print('>> ', end='')
            hour = read_hour()
            print('>> ', end='')
            minute = read_minute()
            print('>> ', end='')
            second = read_second()
            return to_seconds(hour, minute, second)

        print('Start time:')
        start_time = read_time()
        print('Finish time:')
        finish_time = read_time()
print('Elapsed time is', elapsed_time(start_time, finish_time))
```

- [ ] turtle that uses trigonometry

```
from math import degrees,atan
distance = (home.x**2 + home.y**2)**0.5
angle = degrees(atan(abs(home.y/home.x)))
turtle.left(angle)
turtle.forward(distance)
```

- [ ] modulo floor
- [ ] turtle with compass / direction that use angle
- [ ] turtle loop drawing
- [ ] LOOP

Largest Prime Less Than N

Write a Python program that asks the user for a positive integer N. The program then starts from $N - 1$, searchs downward and print the **first prime number** found.

Example 1

```
Enter positive N: 100↵
97
```

Example 2

```
Enter positive N: 97↵
89
```

Example 3

```
Enter positive N: 300↵
293
```

Python Program

[hide line #]

```
n= int(input("Enter positive N: "))
prime=False
n=n-1
while prime == False:
    i=2
    while n%i != 0 and i<n:
        i=i+1
    if i == n:
        prime = True
    else:
        n = n-1

print(n)
```

The Last Product

A factory produces three types of products: \(A\), \(B\), and \(C\).

The production rule is simple: mixing two different types of products always creates one product of the remaining type.

\[A + B \rightarrow C\] \[B + C \rightarrow A\] \[A + C \rightarrow B\]

Each mixing operation consumes one product from each of the two selected types and produces one product of the third type.

For example,

- If the current inventory is: \(A = 13\), \(B = 14\), and \(C = 8\).
- After performing \(A + B \rightarrow C\), the inventory becomes: \(A = 12\), \(B = 13\), and \(C = 9\).

Notice that every mixing operation decreases the total number of products by exactly 1.

To select the best product, the factory repeatedly performs the following process until only one product remains:

1. Choose the **two product types with the largest quantities**.
2. If multiple choices are tied, choose any of them. (It is guaranteed that the final answer is the same regardless of the tie-breaking.)
3. Mix one product from each chosen type according to the rules above.

It is guaranteed that this process always ends with exactly one product remaining.

Write a Python program that accepts the initial quantities of product types \(A\), \(B\), and \(C\), and prints the type of the last remaining product after repeatedly applying the mixing process until only one product remains.

Example 1

```
A: 4
B: 2
C: 3
C
```

Example 2

```
A: 3
B: 4
C: 3
B
```

```
A = int(input("A: "))
B = int(input("B: "))
C = int(input("C: "))

products = [A, B, C]

while sum(products) > 1:
    first = products.index(max(products))
    temp = products[first]
    products[first] = -1
    second = products.index(max(products))
    products[first] = temp

    if first == 0 and second == 1 or first == 1 and second == 0:
        products[0] -= 1
        products[1] -= 1
        products[2] += 1
    elif first == 1 and second == 2 or first == 2 and second == 1:
        products[1] -= 1
        products[2] -= 1
        products[0] += 1
    else:
        products[0] -= 1
        products[2] -= 1
        products[1] += 1

if products[0] == 1:
    print("A")
elif products[1] == 1:
    print("B")
else:
    print("C")
```

☐ string

```python
text = input("Text: ")
pattern = input("Pattern: ")
output = []
output_aprox = []
i = 0

while i < len(text):
    current = text[i:i + len(pattern)]
    if len(current) != len(pattern):
        break
    difference = 0
    for j in range(len(pattern)):
        if current[j] != pattern[j]:
            difference += 1
    if difference == 0:
        output.append([i, current])
        i += len(pattern)
    else:
        i += 1

if len(output) == 0:
    i = 0
    while i < len(text):
        current = text[i:i + len(pattern)]
        if len(current) != len(pattern):
            break
        difference = 0
        different_position = 0
        for j in range(len(pattern)):
            if current[j] != pattern[j]:
                difference += 1
                different_position = j
        if difference == 1:
            holder = list(current)
            holder[different_position] = "?"
            modified = "".join(holder)
            output_aprox.append([i, modified])
            i += len(pattern)
        else:
            i += 1

if len(output) > 0:
    matches = output
else:
    matches = output_aprox

result = ""
last_position = 0

for position, matched in matches:
    result += text[last_position:position]
    result += "[" + matched + "]"
    last_position = position + len(pattern)
result += text[last_position:]

print(result)
```

Approximate Substring

A **substring** is a contiguous sequence of characters within a text.

Given a text and a pattern, your task is to find all occurrences of the pattern in the text using the following rules:

- An **exact match** occurs when every character in the pattern is identical to the corresponding character in the substring.
- An **approximate match** occurs when the substring differs from the pattern in **exactly one character position**.
- Exact matches have higher priority than approximate matches.
- If the text contains one or more exact matches, display **only the exact matches**.
- Approximate matches are considered only when **no exact match exists** in the text.
- For an approximate match, replace the different character with a question mark (`'?'`) before displaying it.

Write a Python program that reads a text and a pattern, then displays the text with every matching substring enclosed in square brackets (`'['` and `']'`).

The text is scanned from left to right. Whenever a match is found, the search continues immediately after the matched substring. Therefore, overlapping matches are not allowed.

You may assume that:

- the pattern is not longer than the text;
- the text and the pattern contain only English letters (`'A'–'Z'` and `'a'–'z'`);
- uppercase and lowercase letters are considered different characters.

Example 1

```
Text: acababababacbabab↵
Pattern: aba↵
ac[aba]b[aba]babcb[aba]b
```

Example 2

```
Text: acababababaccbabab↵
Pattern: abc↵
ac[ab?]b[ab?]b[a?c]b[ab?]b
```

Example 3

```
Text: AAAGTGTGTCTGAC↵
Pattern: GTAT↵
AAA[GT?T][GT?T]GAC
```

- [ ] list (we learn list so we have to use list)
- [ ] file processing

Y1T1FINAL

DATA COLLECTIONS

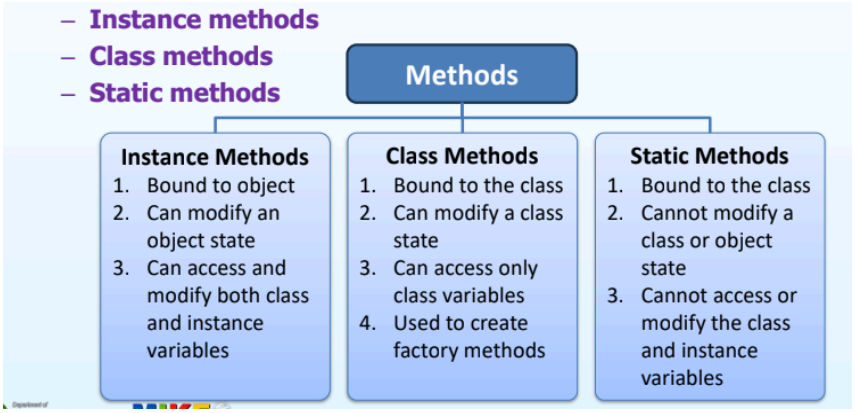
Tuples vs Lists

'''
tuple is immutable, can do slicing [::] , have index
    with one item need (10 ,) or else its (10) number
    where.count('something and index
    can unpack
set is mutable, doesnot repeat, dont havve index
    (use add(one element) update(multiple element) remove(raise error if not exist) discard(no error))
    clear() copy() pop(randomly remove then return that)//takes no arguments
    have operator
        union |
        intersect &
        diff -
        symmetric diff ^ //Find elements in either set, but not in both
dict have key(immutable) and value
    {} is dict

'''
#sorted(data key= lambda x: ??? , reverse = True)

OOP

self = current obj
attribute = instance variable(bound to obj)(each obj has its own copy) / class variable(bound to class)(shared by all obj)
method = instance method(bound to obj) / class method(bound to class) / static method (bound to class) cant modify or access



class method ⇒ use @classmethod

```
class Student:
    school = "ABC School"

    @classmethod
    def change_school(cls, new_school):
        cls.school = new_school
```

static method ⇒ @staticmethod does not depend on a specific object or the class

```
class Calculator:

    @staticmethod
    def add(a, b):
        return a + b
```

Naming	Meaning
name	Public
_name	Protected by convention
__name	Private / name-mangled

```
class Student:
    def __init__(self):
        self.name = "Alice"        # Public
        self._age = 20             # Protected
        self.__password = "1234"   # Private
```

---

encapsulation = bundling / restricting / controlled
name mangling

- A **name mangling** is a way to access private data:

```
account = BankAccount(1000)
print(account._BankAccount__balance)
  # Output: 1000
```

Getters and Setters

- **Getter** is a method used to retrieve a value
- **Setter** is a method used to modify a value, usually with validation

```
class Student:
    def __init__(self, age):
        self.__age = age          # Private

    def get_age(self):
        return self.__age

    def set_age(self, age):
        if age >= 0:    # Prevent invalid data
            self.__age = age

student = Student(20)
print(student.get_age())    # Invoke the Getter
student.set_age(-21)        # Invoke the Setter
```

Pythonic Encapsulation

- Python provides `@property` decorator to make getters and setters easier to use

```
class Student:
    def __init__(self, age):
        self.__age = age          # Private

    @property
    def age(self):
        return self.__age

    @age.setter
    def age(self, age):
        if age >= 0:    # Prevent invalid data
            self.__age = age

student = Student(20)
print(student.age)          # Invoke the Getter
student.age = -21           # Invoke the Setter
```

Using super()

- Sometimes we want to use the parent's implementation and then add something new

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, student_id):
        super().__init__(name)
        self.student_id = student_id

student = Student("Alice", "S001")
print(student.name)         # Output: Alice
print(student.student_id)   # Output: S001
```

inheritance = parent / child

overriding

super()

check inheritance == isinheritance() // bool

Method Resolution Order (MRO) == the order that python searches

polymorphism = "One interface, many forms of behavior"

magic method

- Python uses special methods, called **magic methods** or **dunder methods**, to implement operator behavior

Symbol	Magic Method	Symbol	Magic Method
+	__add__()	%=	__imod__()
-	__sub__()	**=	__ipow__()
*	__mul__()	<	__lt__()
/	__div__()	>	__gt__()
//	__floordiv__()	<=	__le__()
%	__mod__()	>=	__ge__()
**	__pow__()	==	__eq__()
+=	__iadd__()	!=	__ne__()
-=	__isub__()	str()	__str__()
*=	__imul__()	len()	__len__()
/=	__idiv__()		

duck typing

3. Duck Typing

- Python supports polymorphism through **duck typing**

  "If an object behaves like the required type, we can use it."

  although those classes do not inherit from the same class

```
class Dog:
    def speak(self):
        print("Woof!")

class Robot:
    def speak(self):
        print("Beep!")

def make_it_speak(obj):
    obj.speak()

make_it_speak(Dog())    # Output: Woof!
make_it_speak(Robot())  # Output: Beep!
```

Notes:
Four OOP principles:

Encapsulation:
* Refers to bundling data with methods that can operate on that data within a class.
* It's the idea of hiding data within a class, preventing anything outside that class directly interacting with it.
* Keeps the programmer in control of access to data.
* Prevents the programmer from ending up in any strange or unwanted states.

Abstraction:
* Refers to the idea to only showing essential details and keeping everything else hidden.
* The classes you create should act like your car. Users of your classes should not worry about the inner details of those classes.
* This is very important when working on your program incrementally.

Inheritance:
* Is the principle that allows classes to derive from other classes.

Polymorphism:
* Describes methods that are able to take on many forms.
* Allows methods to take on many different forms.
* Be sure that you are calling the correct form of the method.
* There are two types of polymorphism:
** Dynamic polymorphism:
- Occurs during the runtime of the program.
- Describes when a method signature is in both subclass and a superclass.
- The methods share the same name but have different implementation.
- The implementation of the subclass that the object is an instance of overrides that of the superclass.
** Static polymorphism:
- Occurs during compile-time rather than during runtime.
- Refers to when multiply methods with the same name but different arguments are defined in the same class.
- The parameters are different type or taken in different order.
- This is known as method overloading.

Resume:
1) Encapsulation ---> Getters and setters
2) Abstraction --> We care about Interface. Not the implementation behind the hood.
3) Inheritance --> Extending classes... Access modifiers: public, private, modifier
4) Polymorphism --> Interface / Implements in Java ... Same methods' name, but different implementations of methods.

```python
class Book:
    def __init__(self,title, author, is_available =True):
        self.title = title
        self.author = author
        self.is_available = is_available

    def borrow_book(self):
        if self.is_available == True:
            self.is_available = False
            print(f'"{self.title}" has been borrowed.')
        else:
            print(f'"{self.title}" is not available.')
```

```
        return self.is_available

    def return_book(self):
        self.is_available  = True
        print(f'"{self.title}" has been returned.')
        return self.is_available

class Member:
    def __init__(self, name , member_id):
        self.name = name
        self.member_id = member_id
    def borrow_book(self, book):
        print(f'{self.name} is borrowing "{book.title}".')
        return book.borrow_book()
    def return_book(self,book):
        print(f'{self.name} is returning "{book.title}".')
        return book.return_book()

book1 = Book("Python Basics", "John Smith")
member1 = Member("Alice", "M001")

print("Book Information")
print(f"Title: {book1.title}")
print(f"Author: {book1.author}")
print(f"Available: {book1.is_available}")
print()

# Borrow the book
member1.borrow_book(book1)
print(f"Available: {book1.is_available}")
print()

# Try to borrow the book again
member1.borrow_book(book1)
print()

# Return the book
member1.return_book(book1)
print(f"Available: {book1.is_available}")
```

Exception

exception is an event that disrupts the normal flow of a program != error: program cannot start

Exception	Description: Raised when
EOFError	The input() function hits the end-of-file condition
ImportError	The imported module is not found
IndexError	The index of a sequence is out of range
KeyError	A key is not found in a dictionary
NameError	A variable is not found in local or global scope
IndentationError	There is incorrect indentation
TypeError	A function or operation is applied to an object of incorrect type
ValueError	A function gets and argument of correct type but improper value
ZeroDivisionError	The second operand of division or modulo operation is zero

assert statement; check condition if true // AssertionError

```
assert condition, message
```

else block; execute if no exception occurs [only 1]
finally block; always execute [only 1]

```
try:
    # Code that may caused an exception
except ExceptionType:
    # Code to handling the exception
else:
    # Code to execute if no exception occurs
finally:
    # Code to always execute
```

raise statement

```
raise ExceptionType("Error message")
```

“as” keyword; store the exception obj in a variable

```
>>> number = int("abc")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'abc'
>>>
```

```
try:
    number = int("abc")
except ValueError as e:
    print(f"Error: {e}")
```

Sample Outputs:

```
Error: invalid literal for int() with base 10: 'abc'
```

custom exceptions;

Custom Exceptions

- Programmers can create their own exception classes
- A **custom exception** can be defined by inheriting from the built-in **Exception** class

```
class InsufficientBalanceError(Exception):
    pass

balance = int(input("Enter balance: "))
withdraw = int(input("Enter withdraw: "))
try:
    if withdraw > balance:
        raise InsufficientBalanceError("Insufficient balance.")
    balance -= withdraw
    print("Withdrawal successful.")
except InsufficientBalanceError as e:
    print(e)
```

```
Enter balance: 700
Enter withdraw: 500
Withdrawal successful.
```

```
Enter balance: 500
Enter withdraw: 700
Insufficient balance.
```

```
class InvalidAgeError(Exception):
    def __init__(self, age):
        self.age = age
        super().__init__(f"Invalid age: {age}")

    def is_minor(self):
        return self.age < 18

def check_age(age):
    if age < 0 or age > 150:
        raise InvalidAgeError(age)

try:
    check_age(200)
except InvalidAgeError as e:
    print(e)
    print(f"Age: {e.age}")
    print(f"Minor: {e.is_minor()}")
```

Initialize an error message to the (parent) **Exception** class

```
Invalid age: 200
Age: 200
Minor: False
```

Inherit the __str__() method from the (parent) **Exception** class